Synthesis Phase in Compiler Design

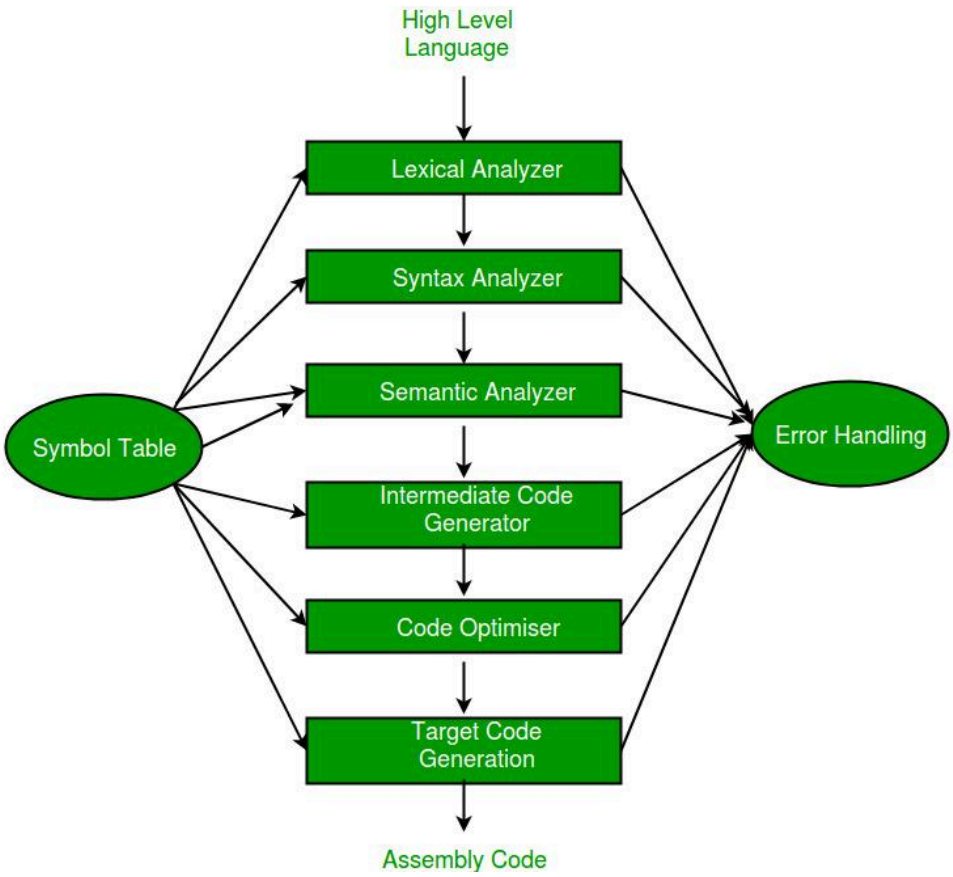
The synthesis phase, also known as the **code generation or code optimization phase,** is the final step of a compiler.

It takes the intermediate code generated by the front end of the compiler and converts it into machine code or assembly code, which can be executed by a computer.

The intermediate code can be in the form of an abstract syntax tree, intermediate representation, or some other form of representation.

The back end of the compiler, which includes the synthesis phase, is responsible for **generating efficient and fast code** by performing various **optimization techniques such as register allocation, instruction scheduling, and memory management.** These optimization techniques are intended to minimize the code size and increase performance by reducing the number of instructions and cycles required for execution.

The output of the synthesis phase is a binary file that can be loaded into memory and executed by the CPU. The generated code is platform-specific and depends on the target architecture that the compiler was designed for. The synthesis phase is crucial for producing efficient and high-performance code that can run on different platforms.



Issues

The synthesis phase, also known as the code generation phase, is the final phase of the compilation process in which the compiler takes the optimized abstract syntax tree (AST) generated in the previous phase and generates machine code or intermediate code that can be executed by the target platform.

There are several potential issues that may arise during the synthesis phase of compilation. Some common issues include:

1. **Code generation errors:** These are errors that occur when the compiler is unable to generate machine code or intermediate code

that is correct or complete. This may be due to errors in the AST or problems with the code generator itself.

2. **Code size and performance:** The compiler may generate machine code that is larger or slower than desired, which can impact the performance of the resulting program.
3. **Compatibility issues:** The generated code may not be compatible with the target platform or with other libraries or frameworks that the program is intended to use.
4. **Linking errors:** If the generated code references external symbols or functions that are not defined, the linker may generate errors when trying to combine the generated code with other object files.

To address these and other issues that may arise during the synthesis phase, compiler designers and developers must carefully design and test their code generators to ensure that they produce high-quality machine code or intermediate code.

Key Terms

Some key important of the synthesis phase include:

1. **Generating machine code:** The synthesis phase takes the intermediate code generated in the previous phase and generates machine code or executable code that can be run on a specific computer architecture.

2. **Improving performance:** The synthesis phase performs various optimization techniques such as instruction selection, register allocation, and memory management to improve the performance of the generated code.
3. **Creating executable files:** The final output of the synthesis phase is typically a file containing machine code or assembly code that can be directly executed by the computer's CPU. This allows the code to be run on the target platform.
4. **Language independent:** As the synthesis phase is typically platform dependent, it can be applied to a wide variety of programming languages and platforms, making it a versatile and reusable component in a compiler.
5. **Enabling optimization:** The synthesis phase uses the information gathered by the analysis phase to perform various optimization techniques to improve the performance of the generated code.
6. **Improving the development process:** By generating high-performance machine code, the synthesis phase can help reduce the need for debugging and testing later on and make the development process more efficient.

Applications

1. **Generating machine code or executable code for a specific platform:** The synthesis phase takes the intermediate code and generates code that can be run on a specific computer architecture.
2. **Instruction selection:** The compiler selects appropriate machine instructions for the target platform to implement the intermediate code.
3. **Register allocation:** The compiler assigns values to registers to improve the performance of the generated code.
4. **Memory management:** The compiler manages the allocation and deallocation of memory to ensure the generated code runs efficiently.
5. **Optimization:** The compiler performs various optimization techniques such as dead code elimination, constant folding, and common subexpression elimination to improve the performance of the generated code.
6. **Creating executable files:** The final output of the synthesis phase is typically a file containing machine code or assembly code that can be directly executed by the computer's CPU.

Introduction to Intermediate Representation(IR)

Last Updated : 7 Jun, 2022

[redacted]

Intermediate Representation(IR), as the name suggests, is any representation of a program between the source and target languages. The intermediate form of the program that is being compiled is the *central data structure* in a compiler. A compiler may have a single IR or a series of IRs. The decisions that are made during the design of IR affect the efficiency and speed of the compiler.

Properties of IRs:

The priorities of different properties across all compilers are not uniform.

The below five are the properties of IRs:

1. Ease of generation
2. Ease of manipulation
3. Freedom of expression
4. Size of the procedure
5. Level of abstraction

Level of Abstraction:

- The amount of details exposed in the IR influence the feasibility and profitability of different optimizations.

- Generally, it can be abstracted into three levels.

  (i) High Level: It is almost similar to the source language. It is good for memory disambiguation. Here, one IR implies one target machine operation. A single IR may include array access or a procedure call. It includes procedure calls and structured objects like structures and arrays.

  (ii) Medium Level: Here, it does not involve any structured objects but still it is independent of the target language. It can be source or target-oriented.

  (iii) Low Level: It is extremely close to the target language. A single IR may include an operation in the procedure. Here several IRs implement one target machine optimization.

Reasons for using Intermediate Representations(IRs):

1. Translating the given code from one form to another requires synthesis and analysis.
2. To perform machine-independent optimizations.
3. To make translation simpler.

Types of IRs:

There are three major categories of Intermediate Representation(IR).

1. Graphical IR:

- It is graphically oriented and it tends to be large. This type of IR is used heavily in source-to-source translators.
- The underlying code is represented as a graph.
- Examples: Directed Acyclic Graphs(DAGs), Trees
- It is further divided into two types:

  (i) Syntax-Related Trees

  (ii) Graphs
- Examples for Syntax-Related trees: Parse Trees, Abstract Syntax Trees(ASTs), Directed Acyclic Graphs(DAGs)
- Examples for Graphs: Control Flow Graphs, Dependency Graphs, and Call Graphs
- Even though all of them contain nodes and edges, they differ in

  (i) Structure of the graph

  (ii) Level of abstraction

  (iii) Relationship between the graph and underlying code

2. Linear IR:

- It is the pseudo-code for an abstract machine. This type of IR contains simple and compact data structures. It is easier to rearrange. The level of abstraction is not uniform, it varies accordingly.

- Examples: Stack machine code, Three address code

3. Hybrid IR:

- As the name suggests, Hybrid IR is the combination of both Graphical IR and Linear IR.

Naming Discipline in IRs:

- While translating the source code to lower-level code, the compiler needs to choose the names for a wide variety of distinct values.
- Example: Consider the expression: x+5*y
- The sequence of operations followed to evaluate the expression are :

  (i) t1<-- y

  (ii) t2<-- 5*t1

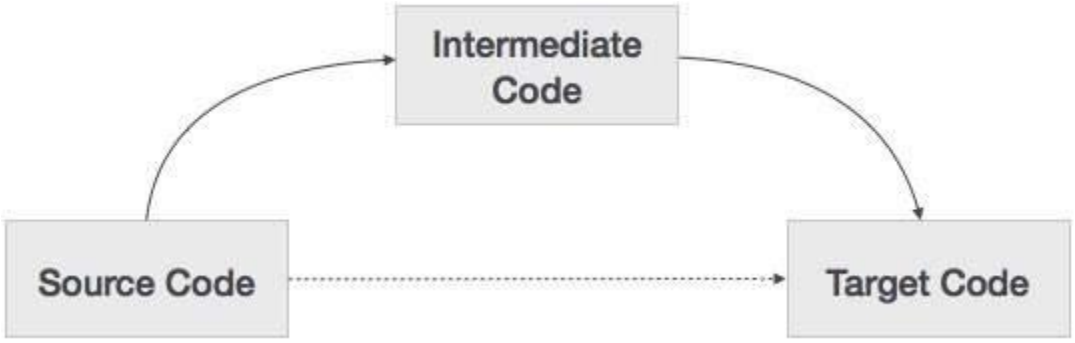
  (iii) t3<--x

  (iv) t4<--t3+t2
- Here the names used are t1,t2,t3, and t4.
- We can also reduce the usage of the number of names by reusing the occurrence of t2 and t4 with t1.
- This operation reduces the number of variables used to half.

- **The naming scheme also has an impact on the compile-time because it determines the sizes of many compile-time data structures.**

Compiler - Intermediate Code Generation

A source code can directly be translated into its target machine code, then why at all we need to translate the source code into an intermediate code which is then translated to its target code? Let us see the reasons why we need an intermediate code.



- If a compiler translates the source language to its target machine language without having the option for generating intermediate code, then for each new machine, a full native compiler is required.
- Intermediate code eliminates the need of a new full compiler for every unique machine by keeping the analysis portion same for all the compilers.
- The second part of compiler, synthesis, is changed according to the target machine.
- It becomes easier to apply the source code modifications to improve code performance by applying code optimization techniques on the intermediate code.

Intermediate Representation

Intermediate codes can be represented in a variety of ways and they have their own benefits.

- High Level IR - High-level intermediate code representation is very close to the source language itself. They can be easily generated from the source code and we can easily apply code modifications to enhance performance. But for target machine optimization, it is less preferred.
- Low Level IR - This one is close to the target machine, which makes it suitable for register and memory allocation, instruction set selection, etc. It is good for machine-dependent optimizations.

Intermediate code can be either language specific (e.g., Byte Code for Java) or language independent (three-address code).

Advertisement

Three-Address Code

Intermediate code generator receives input from its predecessor phase, semantic analyzer, in the form of an annotated syntax tree. That syntax tree then can be converted into a linear representation, e.g., postfix notation. Intermediate code tends to be machine independent code. Therefore, code generator assumes to have unlimited number of memory storage (register) to generate code.

For example:

```
a = b + c * d;
```

The intermediate code generator will try to divide this expression into sub-expressions and then generate the corresponding code.

```
r1 = c * d;
r2 = b + r1;
a = r2
```

r being used as registers in the target program.

A three-address code has at most three address locations to calculate the expression. A three-address code can be represented in two forms : quadruples and triples.

Quadruples

Each instruction in quadruples presentation is divided into four fields: operator, arg1, arg2, and result. The above example is represented below in quadruples format:

Op	arg_1	arg_2	result
*	c	d	r1
+	b	r1	r2
+	r2	r1	r3
=	r3		a

Triples

Each instruction in triples presentation has three fields : op, arg1, and arg2.The results of respective sub-expressions are denoted by the position of expression. Triples represent similarity with DAG and syntax tree. They are equivalent to DAG while representing expressions.

Op	arg_1	arg_2

*	c	d
+	b	(0)
+	(1)	(0)
=	(2)	

Triples face the problem of code immovability while optimization, as the results are positional and changing the order or position of an expression may cause problems.

Indirect Triples

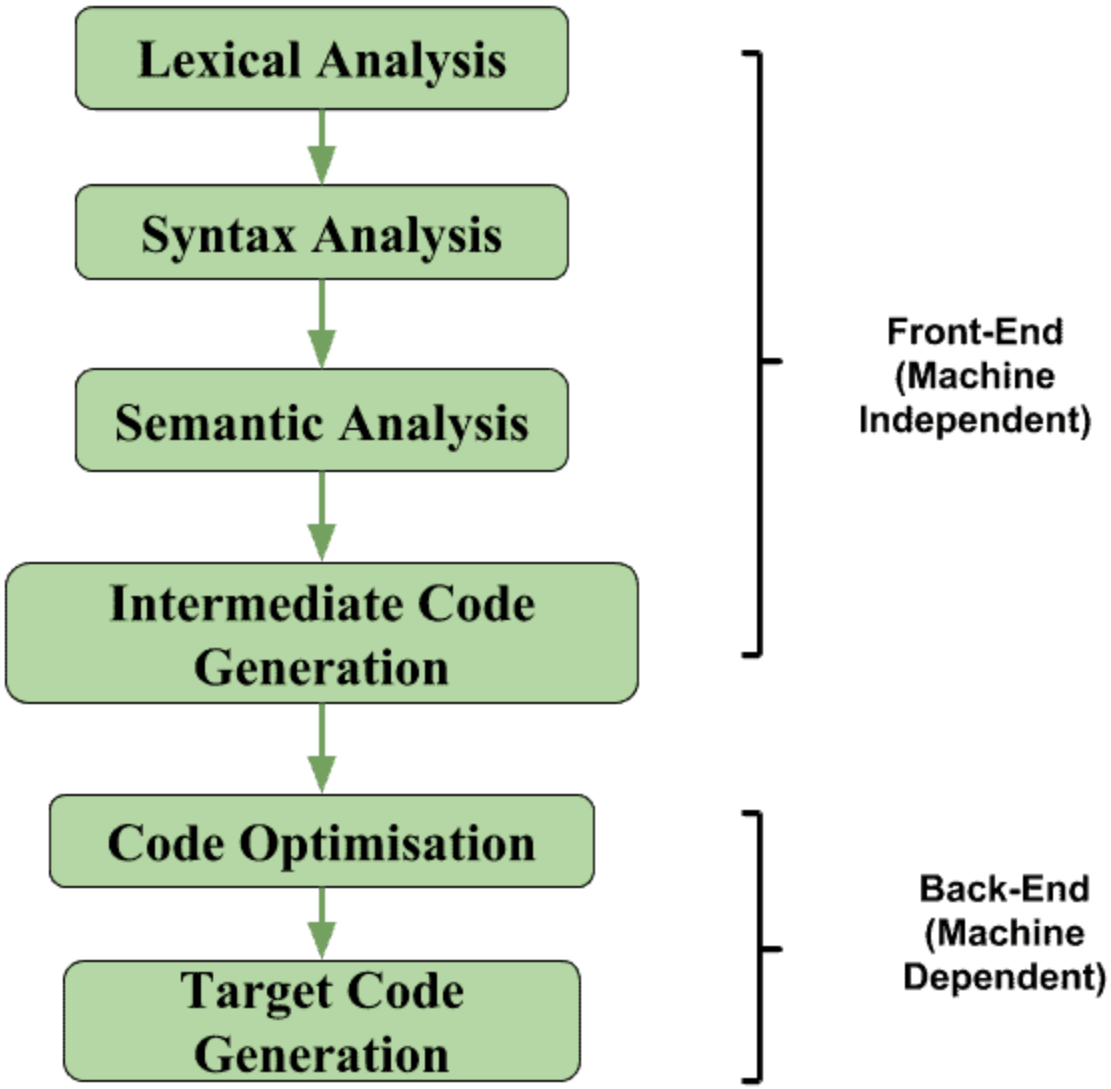
This representation is an enhancement over triples representation. It uses pointers instead of position to store results. This enables the optimizers to freely re-position the sub-expression to produce an optimized code.

Intermediate Code Generation in Compiler Design

Last Updated : 10 Mar, 2026

In the analysis-synthesis model of a compiler, the front end of a compiler translates a source program into an independent intermediate code, then the back end of the compiler uses this intermediate code to generate the target code (which can be understood by the machine). The benefits of using machine-independent intermediate code are:

- Because of the machine-independent intermediate code, portability will be enhanced. For example, suppose, if a compiler translates the source language to its target machine language without having the option for generating intermediate code, then for each new machine, a full native compiler is required. Because, obviously, there were some modifications in the compiler itself according to the machine specifications.
- Retargeting is facilitated.
- It is easier to apply source code modification to improve the performance of source code by optimizing the intermediate code.



Process of Intermediate Code Generation

Intermediate Code Generation is a stage in the process of compiling a program, where the compiler translates the source code into an intermediate representation. This representation is not machine code but is simpler than the original high-level code. Here's how it works:

- **Translation:** The compiler takes the high-level code (like C or Java) and converts it into an intermediate form, which can be easier to analyze and manipulate.
- **Portability**: This intermediate code can often run on different types of machines without needing major changes, making it more versatile.
- **Optimization:** Before turning it into machine code, the compiler can optimize this intermediate code to make the final program run faster or use less memory.

If we generate machine code directly from source code then for n target machine we will have optimizers and n code generator but if we will have a machine-independent intermediate code, we will have only one optimizer. Intermediate code can be either language-specific (e.g., Bytecode for Java) or language. independent (three-address code). The following are commonly used intermediate code representations:

Postfix Notation

- Also known as reverse Polish notation or suffix notation.
- In the infix notation, the operator is placed between operands, e.g., *a + b*. Postfix notation positions the operator at the right end, as in *ab +*.
- For any postfix expressions *e1* and *e2* with a binary operator *(+)* , applying the operator yields *e1e2+*.

- Postfix notation eliminates the need for parentheses, as the operator's position and arity allow unambiguous expression decoding.
- In postfix notation, the operator consistently follows the operand.

Example 1: The postfix representation of the expression (a + b) * c is : ab + c *

Example 2: The postfix representation of the expression (a - b) * (c + d) + (a - b) is : ab - cd + *ab -+

Read more: Infix to Postfix

Three-Address Code

- A three address statement involves a maximum of three references, consisting of two for operands and one for the result.
- A sequence of three address statements collectively forms a three address code.
- The typical form of a three address statement is expressed as *x* = *y op z*, where *x*, *y*, and *z* represent memory addresses.
- Each variable *(x, y, z)* in a three address statement is associated with a specific memory location.

While a standard three address statement includes three references, there are instances where a statement may contain fewer than three references, yet it is still categorized as a three address statement.

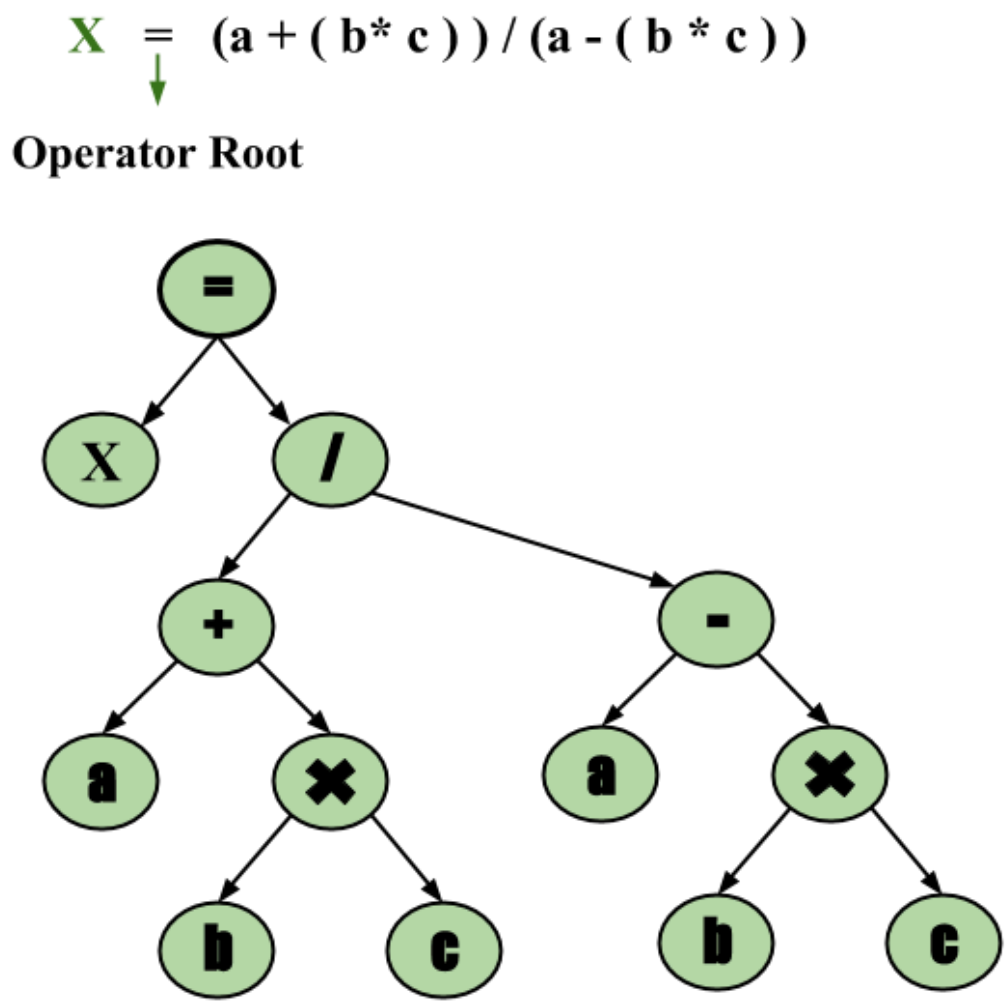
Example: The three address code for the expression a + b * c + d : T1 = b * c T2 = a + T1 T3 = T2 + d; T 1 , T2 , T3 are temporary variables.

There are 3 ways to represent a Three-Address Code in compiler design:
i) Quadruples
ii) Triples
iii) Indirect Triples
Read more: Three-address code

Syntax Tree

- A syntax tree serves as a condensed representation of a parse tree.
- The operator and keyword nodes present in the parse tree undergo a relocation process to become part of their respective parent nodes in the syntax tree. the internal nodes are operators and child nodes are operands.
- Creating a syntax tree involves strategically placing parentheses within the expression. This technique contributes to a more intuitive representation, making it easier to discern the sequence in which operands should be processed.

The syntax tree not only condenses the parse tree but also offers an improved visual representation of the program's syntactic structure,
Example: x = (a + b * c) / (a - b * c)



Key Points:

- Simplifies implementation by reducing the complexity of the input code.
- Enables code optimization for improved performance and efficiency.

- Provides platform independence, allowing the same intermediate code to be used across different machines.
- Facilitates code reuse for generating code for other platforms or languages.
- Makes debugging easier as it is closer to the original source code.
- Can increase compilation time, which may affect real-time or time-critical applications.
- Requires additional memory to store the intermediate representation.
- Adds complexity to the compiler design, making implementation and maintenance harder.
- The generated code may execute slightly slower than code produced directly from source code.

Compiler Design - Code Generation

Code generation can be considered as the final phase of compilation. Through post code generation, optimization process can be applied on the code, but that can be seen as a part of code generation phase itself. The code generated by the compiler is an object code of some lower-level programming language, for example, assembly language. We have seen that the source code written in a higher-level language is transformed into a lower-level language that results in a lower-level object code, which should have the following minimum properties:

- It should carry the exact meaning of the source code.
- It should be efficient in terms of CPU usage and memory management.

We will now see how the intermediate code is transformed into target object code (assembly code, in this case).

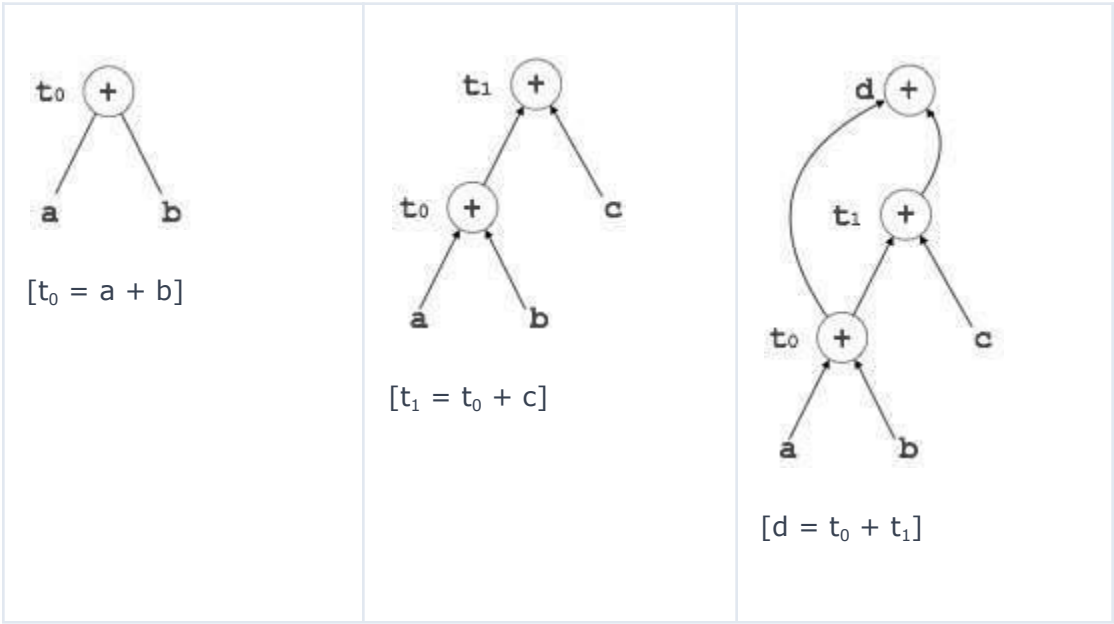
Directed Acyclic Graph

Directed Acyclic Graph (DAG) is a tool that depicts the structure of basic blocks, helps to see the flow of values flowing among the basic blocks, and offers optimization too. DAG provides easy transformation on basic blocks. DAG can be understood here:

- Leaf nodes represent identifiers, names or constants.
- Interior nodes represent operators.
- Interior nodes also represent the results of expressions or the identifiers/name where the values are to be stored or assigned.

Example:

```
t0 = a + b
t1 = t0 + c
d = t0 + t1
```



Advertisement

Peephole Optimization

This optimization technique works locally on the source code to transform it into an optimized code. By locally, we mean a small portion of the code block at hand. These methods can be applied on intermediate codes as well as on target codes. A bunch of statements is analyzed and are checked for the following possible optimization:

Redundant instruction elimination

At source code level, the following can be done by the user:

`int add_ten(int x)`
`{`
`int y, z;`
`y = 10;`
`z = x + y;`
`return z;`
`}`
	`int add_ten(int x)`
	`{`
	`int y;`
	`y = 10;`
	`y = x + y;`
	`return y;`
	`}`
		`int add_ten(int x)`
		`{`
		`int y = 10;`
		`return x + y;`
		`}`
			`int add_ten(int x)`
			`{`
			`return x + 10;`
			`}`

At compilation level, the compiler searches for instructions redundant in nature. Multiple loading and storing of instructions may carry the same meaning even if some of them are removed. For example:

- MOV x, R0
- MOV R0, R1

We can delete the first instruction and re-write the sentence as:

```
MOV x, R1
```

Unreachable code

Unreachable code is a part of the program code that is never accessed because of programming constructs. Programmers may have accidently written a piece of code that can never be reached.

Example:

```
void add_ten(int x)
{
   return x + 10;
   printf(value of x is %d, x);
}
```

In this code segment, the printf statement will never be executed as the program control returns back before it can execute, hence printf can be removed.

Flow of control optimization

There are instances in a code where the program control jumps back and forth without performing any significant task. These jumps can be removed. Consider the following chunk of code:

```
...
MOV R1, R2
GOTO L1
...
L1 :   GOTO L2
L2 :   INC R1
```

In this code,label L1 can be removed as it passes the control to L2. So instead of jumping to L1 and then to L2, the control can directly reach L2, as shown below:

```
...
MOV R1, R2
GOTO L2
...
L2 :   INC R1
```

Algebraic expression simplification

There are occasions where algebraic expressions can be made simple. For example, the expression a = a + 0 can be replaced by a itself and the expression a = a + 1 can simply be replaced by INC a.

Strength reduction

There are operations that consume more time and space. Their strength can be reduced by replacing them with other operations that consume less time and space, but produce the same result.

For example, x * 2 can be replaced by x , which involves only one left shift. Though the output of a * a and a^2 is same, a^2 is much more efficient to implement.

Accessing machine instructions

The target machine can deploy more sophisticated instructions, which can have the capability to perform specific operations much efficiently. If the target code can accommodate those instructions directly, that will not only improve the quality of code, but also yield more efficient results.

Code Generator

A code generator is expected to have an understanding of the target machines runtime environment and its instruction set. The code generator should take the following things into consideration to generate the code:

- Target language : The code generator has to be aware of the nature of the target language for which the code is to be transformed. That language may facilitate some machine-specific instructions to help the compiler generate the code in a more convenient way. The target machine can have either CISC or RISC processor architecture.
- IR Type : Intermediate representation has various forms. It can be in Abstract Syntax Tree (AST) structure, Reverse Polish Notation, or 3-address code.
- Selection of instruction : The code generator takes Intermediate Representation as input and converts (maps) it into target machines

instruction set. One representation can have many ways (instructions) to convert it, so it becomes the responsibility of the code generator to choose the appropriate instructions wisely.

- Register allocation : A program has a number of values to be maintained during the execution. The target machines architecture may not allow all of the values to be kept in the CPU memory or registers. Code generator decides what values to keep in the registers. Also, it decides the registers to be used to keep these values.
- Ordering of instructions : At last, the code generator decides the order in which the instruction will be executed. It creates schedules for instructions to execute them.

Descriptors

The code generator has to track both the registers (for availability) and addresses (location of values) while generating the code. For both of them, the following two descriptors are used:

- Register descriptor : Register descriptor is used to inform the code generator about the availability of registers. Register descriptor keeps track of values stored in each register. Whenever a new register is required during code generation, this descriptor is consulted for register availability.
- Address descriptor : Values of the names (identifiers) used in the program might be stored at different locations while in execution. Address descriptors are used to keep track of memory locations where the values of identifiers are stored. These locations may include CPU registers, heaps, stacks, memory or a combination of the mentioned locations.

Code generator keeps both the descriptor updated in real-time. For a load statement, LD R1, x, the code generator:

- `updates the Register Descriptor R1 that has value of x and`

- `updates the Address Descriptor (x) to show that one instance of x is in R1.`

Code Generation

Basic blocks comprise of a sequence of three-address instructions. Code generator takes these sequence of instructions as input.

Note : If the value of a name is found at more than one place (register, cache, or memory), the registers value will be preferred over the cache and main memory. Likewise caches value will be preferred over the main memory. Main memory is barely given any preference.

getReg : Code generator uses *getReg* function to determine the status of available registers and the location of name values. *getReg* works as follows:

- If variable Y is already in register R, it uses that register.
- Else if some register R is available, it uses that register.
- Else if both the above options are not possible, it chooses a register that requires minimal number of load and store instructions.

For an instruction x = y OP z, the code generator may perform the following actions. Let us assume that L is the location (preferably register) where the output of y OP z is to be saved:

- Call function getReg, to decide the location of L.
- Determine the present location (register or memory) of y by consulting the Address Descriptor of y. If y is not presently in register L, then generate the following instruction to copy the value of y to L:
  MOV y, L
  where y represents the copied value of y.
- Determine the present location of z using the same method used in step 2 for y and generate the following instruction:
  OP z, L
  where z represents the copied value of z.

- Now L contains the value of y OP z, that is intended to be assigned to x. So, if L is a register, update its descriptor to indicate that it contains the value of x. Update the descriptor of x to indicate that it is stored at location L.
- If y and z has no further use, they can be given back to the system.

Other code constructs like loops and conditional statements are transformed into assembly language in general assembly way.

Code Optimization in Compiler Design

Last Updated : 10 Mar, 2026

Code optimization is the process of improving a program to make it more efficient in terms of speed, memory, and resource usage, without changing its functionality. The key aspects of code optimization include:

- **Improved performance:** Optimized code executes faster and uses fewer resources.
- **Reduced code size:** Smaller code is easier to distribute and deploy.
- **Increased portability:** Optimized code can run efficiently on multiple platforms.
- **Lower power consumption:** Efficient code uses less energy, making it more energy-friendly.
- **Better maintainability:** Well-optimized code can be easier to read and maintain.

- **Reasonable compilation time:** Optimization should not make compilation excessively slow.
- **Correctness assurance:** Optimization must preserve the program's original behavior and logic.
- **Manageable complexity:** Code optimization may increase complexity, but the program should remain easy to understand, maintain, and debug.

When to Optimize?

Optimization of the code is often performed at the end of the development stage since it reduces readability and adds code that is used to increase performance.

Why Optimize?

Algorithm optimization is beyond the scope of the code optimization phase. Instead, the compiler optimizes the intermediate program code, which may also reduce the size of the generated code. Therefore, code optimization helps to:

- Reduce memory usage and code size.
- Improve program execution speed.
- Automatically optimize code using a compiler optimizer, since manual optimization is difficult and time-consuming.

Types of Code Optimization

- **Machine Independent Optimization:** Improves the intermediate code to produce more efficient target code. It is performed without considering the target machine architecture. The transformations do not involve CPU registers or absolute memory addresses.
- **Machine Dependent Optimization:** Performed after the target code is generated. It optimizes the code according to the specific architecture of the target machine. This type of optimization uses CPU registers and may involve absolute memory references to take better advantage of the memory hierarchy and hardware features.

Ways to Optimize Code

There are several ways to optimize code. Some of them are mentioned below.

1. Compile Time Evaluation:

Expressions evaluated during compilation.

```
// Before Optimization
A = 2 * (22.0 / 7.0) * r

// After Optimization
A = 6.2857 * r
```

2. Variable Propagation:

Replaces a variable with another variable having the same value.

```
// Before Optimization
x = a
y = x + 5

// After Optimization
y = a + 5
```

3. Constant Propagation:

Replaces variables with their constant values.

Example:

```
// Before Optimization
a = 10
b = a + 5

// After Optimization
a = 10
b = 10 + 5
```

4. Constant Folding:

Computes constant expressions at compile time.

Example:

```
#define k 5
x = 2 * k
y = k + 5
 This can be computed at compile time and the values of x and y are
:
x = 10
```

```
y = 10
```

5. Copy Propagation:

- It is extension of constant propagation.
- Replaces copied variables with the original variable.

Example :

```
//Before Optimization
 c = a * b
 x = a
 till
 d = x * b + 4


 //After Optimization
 d = a * b + 4
```

6. Common Sub Expression Elimination:

Avoids repeated calculation of the same expression.

```
// Before Optimization
t1 = a * b
t2 = a * b + c

// After Optimization
t1 = a * b
t2 = t1 + c
```

7. Dead Code Elimination:

Removes code that is never used.

Example:

```
// Before Optimization
x = 10
y = 20
print(y)

// After Optimization
y = 20
print(y)
```

8. Unreachable Code Elimination:

Removes code that will never execute.

```
#include <iostream>
using namespace std;

int main() {
 int num;
 num=10;
   cout << "GFG!";
   return 0;
 cout << num; //unreachable code
}
//after elimination of unreachable code
int main() {
 int num;
 num=10;
   cout << "GFG!";
   return 0;
}
```

9. Function Inlining:

Replaces a function call with its body.

```
// Before Optimization
sum(a,b){
  return a + b
}
x = sum(2,3)

// After Optimization
x = 2 + 3
```

10. Function Cloning:

- Here, specialized codes for a function are created for different calling parameters.
- **Example:** Function Overloading

```
// Before Optimization
int add(int a, int b){
   return a + b;
}

int x = add(2,3);
float y = add(4,5);
// After Optimization (Function Cloning)
int add_int(int a, int b){
   return a + b;
}

float add_float(float a, float b){
   return a + b;
}

int x = add_int(2,3);
float y = add_float(4,5);
```

11. Induction Variable :

Uses variables that change regularly in loops.

Examples:

```
// Before Optimization
for(i = 0; i < n; i++){
   x = 4 * i;
   print(x);
}
// After Optimization
x = 0;
for(i = 0; i < n; i++){
   print(x);
   x = x + 4;
}
```

12. Strength Reduction:

Replaces costly operations with cheaper ones.

```
// Before Optimization
x = y * 2

// After Optimization
x = y + y
```

Loop Optimization Techniques

1. Code Motion (Frequency Reduction)

This technique reduces the number of times an expression is evaluated by moving **loop-invariant statements outside the loop**.

Example:

```
// Before Optimization
for(i = 0; i < n; i++){
    x = a * b;
    y[i] = x + i;
}

// After Optimization
x = a * b;
for(i = 0; i < n; i++){
    y[i] = x + i;
}
```

2. Loop Jamming

Two or more loops are combined in a single loop to helps in reducing the compile time.

Example:

```
// Before Optimization
for(i = 0; i < n; i++){
   a[i] = b[i] + c[i];
}

for(i = 0; i < n; i++){
   d[i] = e[i] + f[i];
}

// After Optimization
for(i = 0; i < n; i++){
   a[i] = b[i] + c[i];
```

```
   d[i] = e[i] + f[i];
}
```

3. Loop Unrolling

This technique reduces the number of loop iterations by performing multiple operations in a single iteration, which decreases loop control overhead.

Example:

```
// Before Optimization
for(i = 0; i < 4; i++){
    a[i] = a[i] * 2;
}

// After Optimization
a[0] = a[0] * 2;
a[1] = a[1] * 2;
a[2] = a[2] * 2;
a[3] = a[3] * 2;
```

Where to Apply Optimization?

Code optimization can be applied at different stages of program execution and compilation to improve performance and efficiency.

1. Source Program Level

Optimization at the source program level involves modifying the algorithm or program structure, such as improving loop structures or reducing unnecessary computations. Here, the programmer (user) performs the optimization.

2. Intermediate Code Level

At this stage, the compiler optimizes the intermediate representation of the program. This may include improving address calculations and optimizing procedure calls.

3. Target Code Level

Optimization is performed on the generated machine code. The compiler improves the use of CPU registers, instruction selection, and instruction movement to produce efficient target code.

Optimization Based on Program Scope

Type	Description	Techniques Used
Local Optimization	Applied to small basic blocks of statements.	Local Value Numbering, Tree Height Balancing
Regional Optimization	Applied to Extended Basic Blocks.	Super Local Value Numbering, Loop Unrolling

Global Optimization	Applied to large program sections such as functions and loops.	Live Variable Analysis, Global Code Replacement
Interprocedural Optimization	Applied across multiple procedures or functions.	Inline Substitution, Procedure Placement